

2.2.1 – 2.2.2 Modularity and Readability

ماڈیولریٹی کا مطلب الگورتھم کو آزاد حصوں (فنکشن، کلاس، ماڈیول) میں توڑنا ہے۔

CS XII

Chapter 2: Computational Thinking & Algorithms

Lecture 04 of 16

2.2.1 – 2.2.2 Modularity and Readability

LEARN IT

Modularity means breaking an algorithm into smaller, independent parts such as **functions, classes or modules**. This makes the algorithm more organised and easier to work with.

How to evaluate modularity

1. **Single Responsibility Principle (SRP)**. Each module should focus on **one** specific task. If a function calculates a sum, prints a report and saves a file, it violates SRP.
2. **Coupling**. Modules should be reusable in different contexts. Good modularity means **loose coupling**: a change in one part does not smash the others. Tight coupling is when two functions share so many hidden details that you cannot reuse one without the other.
3. **Abstraction**. A well-designed algorithm hides complex implementation details. You can call a sorting function without knowing whether it uses Bubble Sort or Selection Sort inside.

WORKED EXAMPLE

Worked example: average of an array.

Poor modularity (monoblock)

All logic sits in one block, so you cannot reuse the sum.

Step 1: Start
Step 2: Set A = [1, 2, 3, 4, 5]
Step 3: Set n = 5
Step 4: Set total = 0
Step 5: Set avg = 0
Step 6: For I = 1 to n
Step 7: total = total + A[I]
Step 8: End For
Step 9: avg = total / n
Step 10: Print avg
Step 11: End

Good modularity (functions)

Two clear functions: `CalculateSum()` and `FindAverage()`.

Step 1: Start
Step 2: Set A = [1, 2, 3, 4, 5]
Step 3: Set n = 5
Step 4: Print FindAverage(A, n)
Step 5: End

```
Function FindAverage(A, n)
    Sum = CalculateSum(A, n)
    Avg = Sum / n
    Return Avg
End Function
```

```
Function CalculateSum(A, n)
    Set Total = 0
    For I = 1 to n
        Total = Total + A[I]
    End For
    Return Total
End Function
```

SRP check: `CalculateSum` only adds; `FindAverage` only divides. Loose coupling: you can reuse `CalculateSum` to find a total without computing an average. Abstraction: the main algorithm just calls `FindAverage` and does not see the loop.

The source pseudocode numbers array positions from 1. Python numbers them from 0, so the loop is `for i in range(n)`.

```
def calculate_sum(A, n):
    total = 0
    for i in range(n):
        total = total + A[i]
    return total

def find_average(A, n):
    return calculate_sum(A, n) / n

A = [1, 2, 3, 4, 5]
n = 5
print(find_average(A, n))
```

OUTPUT
3.0

REMEMBER IT

SRP in one line: **one function, one job**. If you cannot name the function with a short verb-phrase (`CalculateSum`, `FindAverage`), it is doing too much.

COMMON MISTAKE

Mistake: copying the 1-based loop `for I = 1 to n: total = total + A[I]` into Python. Then `A[1]` skips the first item and `A[5]` crashes.

Fix: in Python use `for i in range(n)` so the indices are 0, 1, 2, 3, 4.

اردو میں سمجھیں

ماڈیولریٹی کا مطلب الگورتھم کو آزاد حصوں (فنکشن، کلاس، ماڈیول) میں توڑنا ہے۔ جانچ کے تین اصول: واحد ذمہ داری (SRP) یعنی ایک فنکشن ایک کام؛ ڈھیلی جڑت (Loose Coupling) تاکہ ایک حصہ بدلنے سے دوسرے نہ ٹوٹیں؛ اور تجرید (Abstraction) یعنی اندرونی تفصیل چھپا کر صرف کام منگوانا، جیسے سورٹ فنکشن بلائے بغیر یہ جانے کہ ببل ہے یا سلیکشن۔

CHECK YOURSELF

1. What does SRP say each module should do?
2. In the good average algorithm, which function hides the loop?
3. Why is the monoblock harder to reuse?

Answers: 1. One specific task. 2. `CalculateSum` (`FindAverage` only divides; the main program only prints). 3. All logic is in one block, so you cannot call "just the sum" from another algorithm.

LEARN IT

Readability is how easily another person can understand the purpose and logic of an algorithm **without running it**. It is about how clear the text and the flow are.

How to evaluate readability

1. **Naming conventions** — consistent rules for names. Good names make an algorithm easier to search and understand.
Bad: $v = d / t$
Good: `velocity = distance / time`
2. **Self-documentation** — the algorithm explains itself, so you need fewer comments. Meaningful names and clean formatting make the purpose obvious.
3. **Control-flow simplicity** — keep the logic straightforward. Avoid deep nesting (IF inside IF inside IF), limit extra conditions, and keep each function on one task.
4. **Consistency** — one style throughout: indentation, spacing, and casing (camelCase or snake_case). Then the logic stands out instead of the mess.

Names → Self-doc → Simple flow → Consistent

WORKED EXAMPLE

Worked example: the same formula, two styles.

```
# Hard to read
d = 100
t = 4
s = 20
v = d / t
if v > s:
    if t > 0:
        print(v)
```

OUTPUT
25.0

```
# Readable
distance = 100
time = 4
velocity = distance / time
speed_limit = 20
if time > 0 and velocity > speed_limit:
    print(velocity)
```

OUTPUT
25.0

The second version uses names, avoids an IF nested inside an IF, and keeps one consistent snake_case style. You can understand it without running it. Both print 25.0, but only the second is clear.

REMEMBER IT

Remember readability as **N-S-S-C**: **N**ames, **S**elf-documentation, **S**imple flow, **C**onsistency.

COMMON MISTAKE

Mistake: dumping a comment on every line (`total = total + 1 # add one to total`).

Fix: if the name already says it, skip the comment. Comments should explain **why**, not repeat **what**.

پڑھائی پن یہ ہے کہ دوسرا شخص الگورتھم چلائے بغیر اس کا مقصد سمجھ لے۔ اچھے نام ($velocity = distance / time$)، خود بیان (کم تبصرے)، سادہ کنٹرول فلو (گہری IF کے بغیر) اور ایک جیسا انداز (casing اور indentation) اسے آسان بناتے ہیں۔

CHECK YOURSELF

1. Rewrite $v = d / t$ with readable names.
2. What is deep nesting, and why is it a readability problem?
3. Name two style habits that support consistency.

Answers: 1. $velocity = distance / time$. 2. IF inside IF inside IF (or similar); it makes the flow hard to follow. 3. Any two of: consistent indentation, spacing, and casing (camelCase or snake_case).

Extra board practice

These MCQs and questions are extra to the textbook. Close the notes, attempt them, then open the key.

MCQ 1. Modularity is:

- a. Splitting into independent pieces with clear interfaces
- b. One file of 2000 mixed lines
- c. Random copy-paste
- d. A chart type

MCQ 2. A module should have:

- a. One job
- b. Every job
- c. No name
- d. No inputs

MCQ 3. Readability is:

- a. How easily another person understands the algorithm
- b. Font size of the PDF only
- c. RAM size
- d. Cable length

MCQ 4. Reusing a module is easier if:

- a. It does not depend on hidden globals
- b. It prints inside and uses global n
- c. It is unnamed
- d. It mixes I/O and maths

MCQ 5. Comments should explain:

- a. Why, not every obvious $i=i+1$
- b. Nothing
- c. Only your WhatsApp
- d. Only Big O symbols

MCQ 6. Poor readability increases:

- a. Bugs when changing code
- b. CPU GHz
- c. OSI layers
- d. Pie slices

Extra short questions

1. Define modularity and readability.
2. One benefit of each.
3. A readable name vs a bad one.
4. How they interact.

Extra long question

1. Take a messy attendance script (globals, 1-letter names). Produce a modular readable version and a 5-point rubric a teacher could mark.

Extra answer key

MCQs: 1. A 2. A 3. A 4. A 5. A 6. A

Short

1. Modularity: parts. Readability: understandable text/structure.
2. Modularity: test/reuse. Readability: fewer maintenance bugs.
3. total_marks vs x1.
4. Small named modules are easier to read.

Long

1. Modules: load, mark, save, report. Rubric: names, functions, no magic numbers, comments why, handles empty file.

Next lecture: 2.3 Assess Algorithm Efficiency